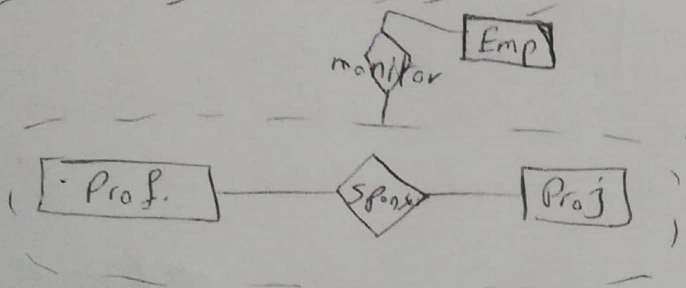
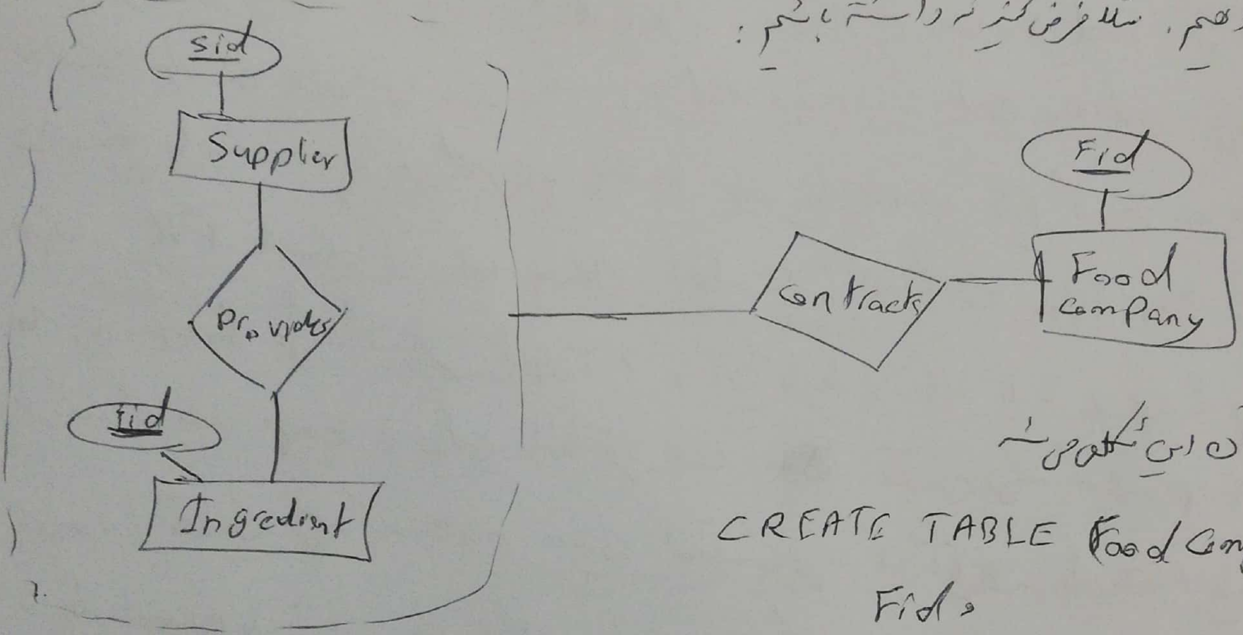
علیرضا کریمی ۸۱۰۱۰۱۴۹۲

الف) باید به طور کلی اول aggregation رو یک entity در نظر بگیریم و سپس یک entity باهاش رابطه برقرار کنیم. و از طرفی یک راه دیگر هم هست که مثلا اگر داشته باشیم (



) در این صورت، برای table مربوط به Emp_monitor باید یک refrence بزنیم به کلاس مربوط به Sponsor و foreign key آن را دقیقاً همان Primary key این table مربوط به sponsor قرار می دهیم. مثلا فرض کنید که داشته باشیم:



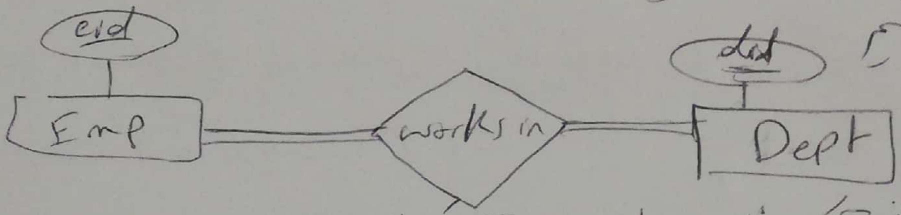
کد مربوط به آن این شکلی میشه

```
CREATE TABLE Suppliers(
        sid,
        PK(sid))
CREATE TABLE Ingredients(
    iid,
    PK(iid))
CREATE TABLE Provides(
    sid,
    iid,
    PK(sid, iid)
    FK(iid) REFRENCE Ingredients ON DELETE NO ACTION
    FK(sid)     "     Suppliers  "   "   "   "   )
```

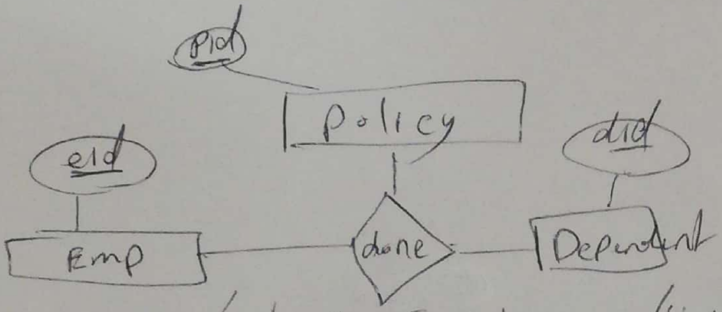
```
CREATE TABLE FoodCompany(
        Fid,
        PK(Fid))
CREATE TABLE Contracts(
    iid,
    sid,
    fid,
    PK(sid, iid, fid)
★   FK(sid, iid) Refrence Provides
    FK(fid) REFRENCE FoodCompany)
```

نکته aggregation ↑

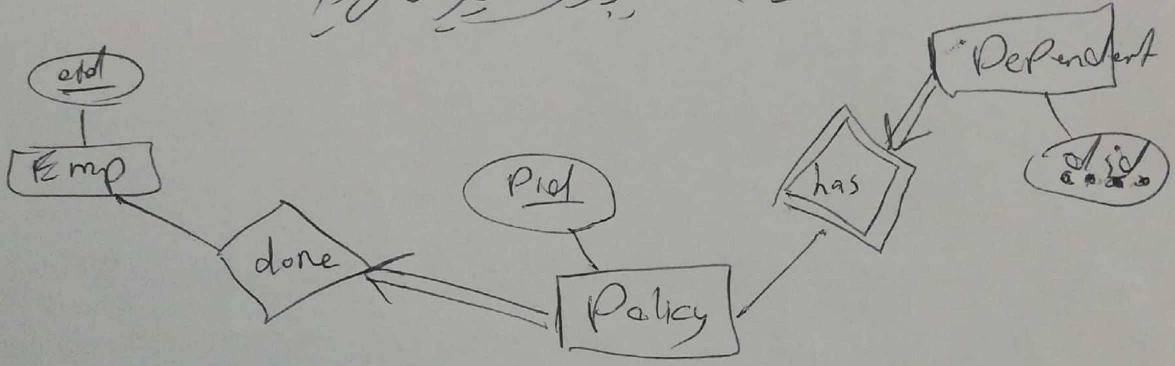
1) ب) مشکلش اینه که ما نمی‌تونیم یک آرایه‌ای که حداقل 1 تا هست رو تضمین کنیم که می‌تونه ذخیره کنه.
مثلا فرض کنید داشته باشیم



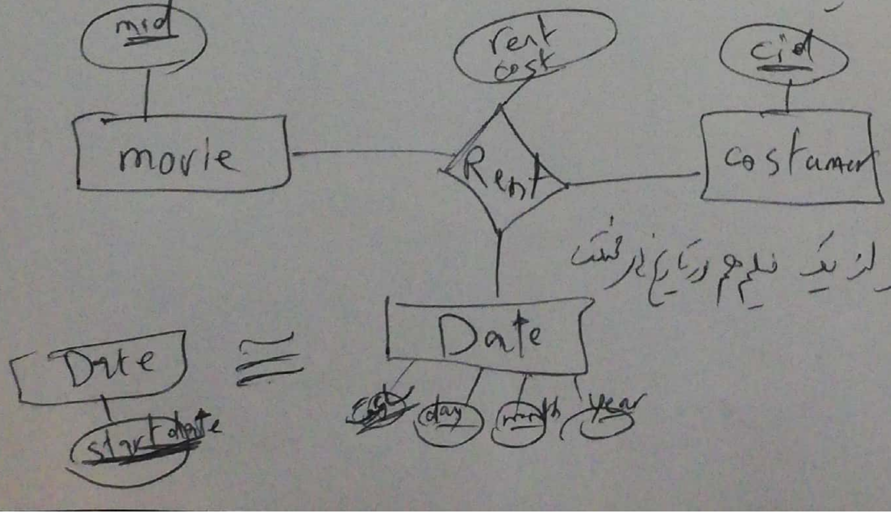
خب توی این حالت، اینطوری نیست که مثلا works in بره توی یکی از table های Emp یا Dept،
چرا؟ چون خودش یک موجودیت جداست. (زمانی که محدودیت key constraint داشتیم را گفتیم
که works in باید توی Dept مثلا و هر did خودش یک به صورت Primary Key
کافیه و اینطور نمیشد که PK(eid,did) داشته باشیم، چون هر did، دوتا eid نمیت
نداشت براش ولی اینجا دقیقا برعکسه و هر did می‌تونه چندتا eid و هر eid می‌تونه چندتا did
داشته باشه پس یک table برای works in داریم که حتما باید هر دو ستون
جزو PK باشن) اما مشکل اینجاست. جدا کردن table مربوط به رابطه works in
ها رو force نمی‌کنه که همه Dept ها در این رابطه قرار دارن یا ما رو force نمی‌کنه و چک
نمی‌کنه که همه Emp ها در این رابطه قرار دارن. تضمین نمی‌کنه که همه Emp ها یا Dept ها
اومده (تضمین می‌کنه که null نیستن ولی تضمین نمی‌کنه که آیا همه Dept ها یا Emp ها در این
رابطه اومدن یا نه) یعنی امکان این رو میده که Emp یا Dept ای باشه که در رابطه نباشه
و table مربوط به works in، اینا رو چک نمی‌کنه.



۱) ج) مثلا مثال زیر را در نظر بگیرید:
حال فرض کنید بخواهیم یک سری محدودیت ها و یک سری تغییرات سمانتیک اضافه کنیم. مثلا اینکه هر Policy دقیقا توسط یک Emp انجام می‌شه، خب می‌تونیم با فلش پررنگ که Policy به relation این رو مشخص کنیم (چون معنی اش اینه که هر Policy دقیقا با یک Emp و دقیقا با یک Dependent انجام می‌شه) حال فرض کنید که محدودیت های دیگه هم داریم. هر Dependent دقیقا برای یک Policy هست (و فرض کنید که می‌خوایم Dependent ها weak entity هم باشه، دیگه اصلا با این مثال ternary نمی‌شه و باید حتما تغییرش بدیم.) اینجوری تغییرش می‌دیم



۲) الف) مشکلش اینه که اگه یک فرد، یک movie رو بیش از چند بار Rent کنه، فقط تاریخ آخر در اول ثبت می‌شه به جای تاریخ های قبل (یجوری تاریخ های قبلی پاک می‌شه اگه update بشه با جدیده) اما شاید ما بخوایم همه تاریخ ها رو داشته باشیم. برای رفع این مشکل، Date رو یک entity می‌کنیم و رابطه مون رو ternary می‌کنیم به جای binary



اینجوری برای Rent های مختلف یک نفر از یک فیلم هم در تاریخ های مختلف می‌تونیم قیمت های مختلف بزاریم

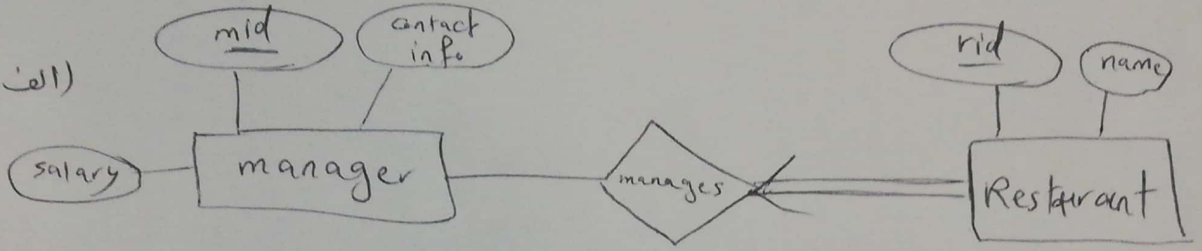
۲ ب)

```
CREATE VIEW ALI_VIEW(movie_name, director_name)
  AS SELECT M.movie_name, M.director_name
  FROM Customer C, Rents R, Movie M
  WHERE C.name = 'Ali' AND R.rent_start_date > 1999
```

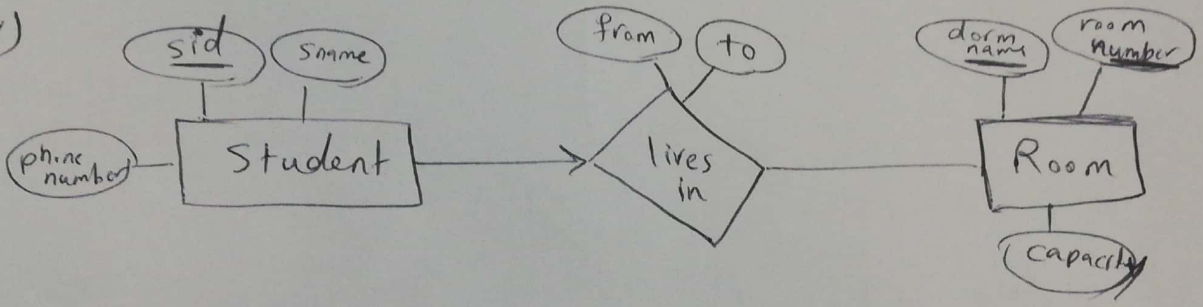
۲) ج) کلا داستان داریم دیگه، یجور تغییر این view ها مثل تغییر cash می‌مونه که شاید در حافظه اصلی که همون base table های ما هستن، تغییرات اعمال نشه. یعنی مثلا اگر بخوایم یک ردیف از یک view حذف کنیم، چون ممکن است از چند table مختلف انتخاب کرده باشیم، با حذف اون ردیف معلوم نیست کدام base table های ما تغییر کنن و این تغییر به چه شکلی باشد. شاید به اشتباه بخواهد field ها (ستون‌های) جدیدی با مقدار NULL به بعضی از base table ها اضافه کند چون که اون field ها در base table نبودن اما در view بوده (چون در view از چند تا base table کمک گرفته بودیم. گاهی اوقات حتی همین update کردن view هم به درستی انجام نمی‌شود، چون RRIMARY KEY ندارن. نبوده و اون یکتا بودن زیاد رعایت نشده و ممکن است چند تا id متفاوت، اطلاعات دیگه‌شون یکی بوده و همون اطلاعات دیگه در view اومده و update اون view شاید از مد نظر ما، باید برای یکی از اون چند تا id اعمال می‌شده که این اتفاق نمی‌افته.

3)

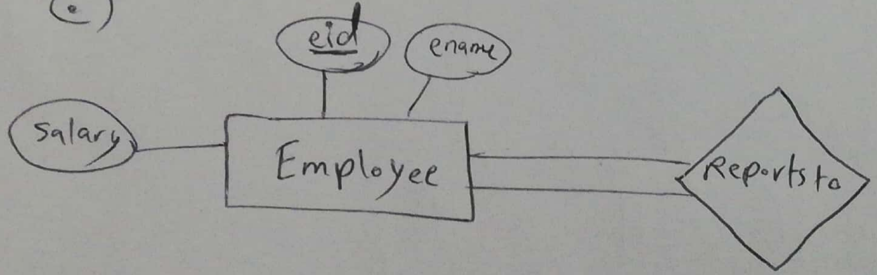
الف)



ب)

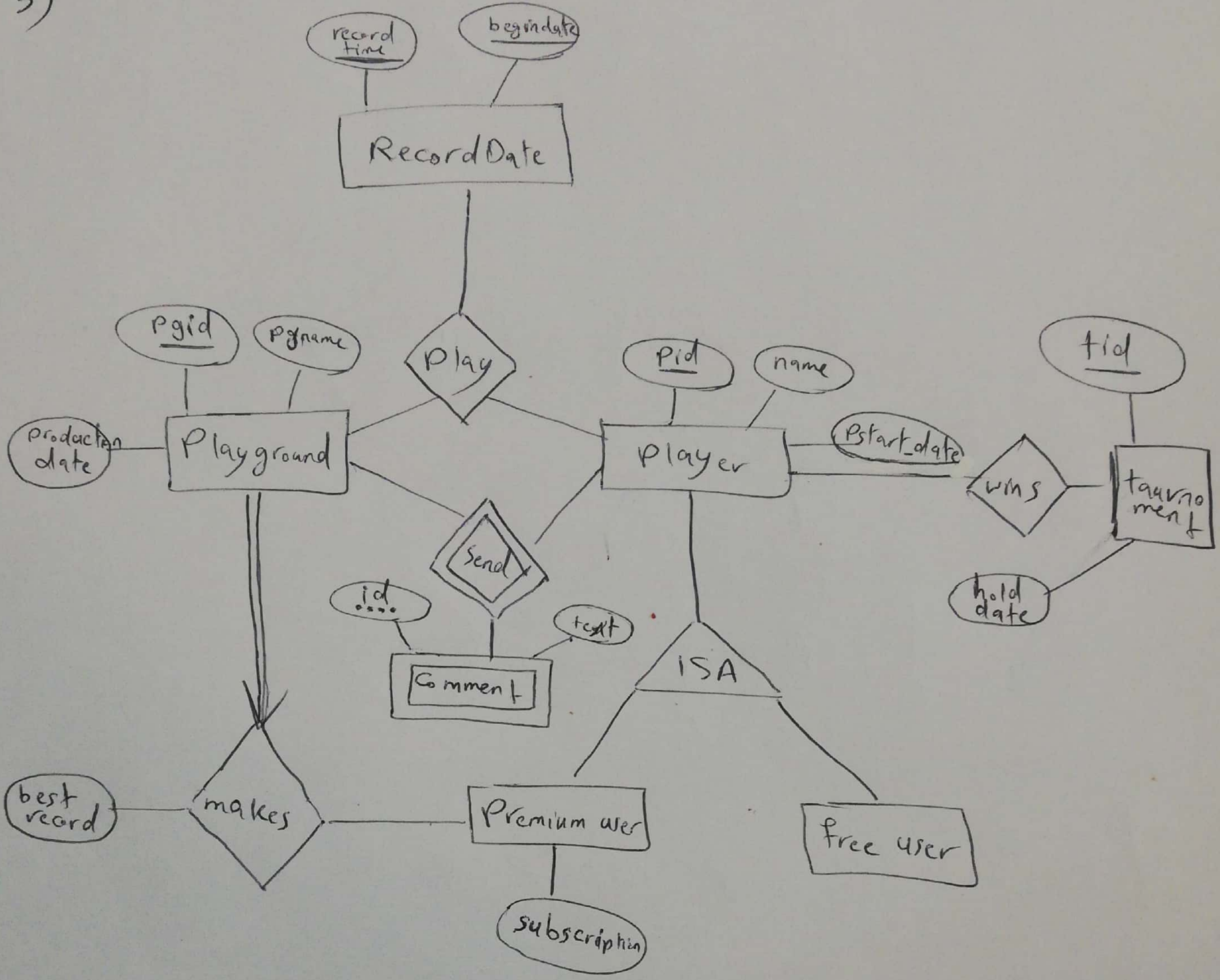


ج)



۴) کد این سوال رو به همراه فایل zip شده می‌فرستم.

5)



CamScanner